LINUX DAY · MODENA 24/10/2026

VPN in ambiente Linux

Tunnel, incapsulamento e cifratura, con una dimostrazione dal vivo

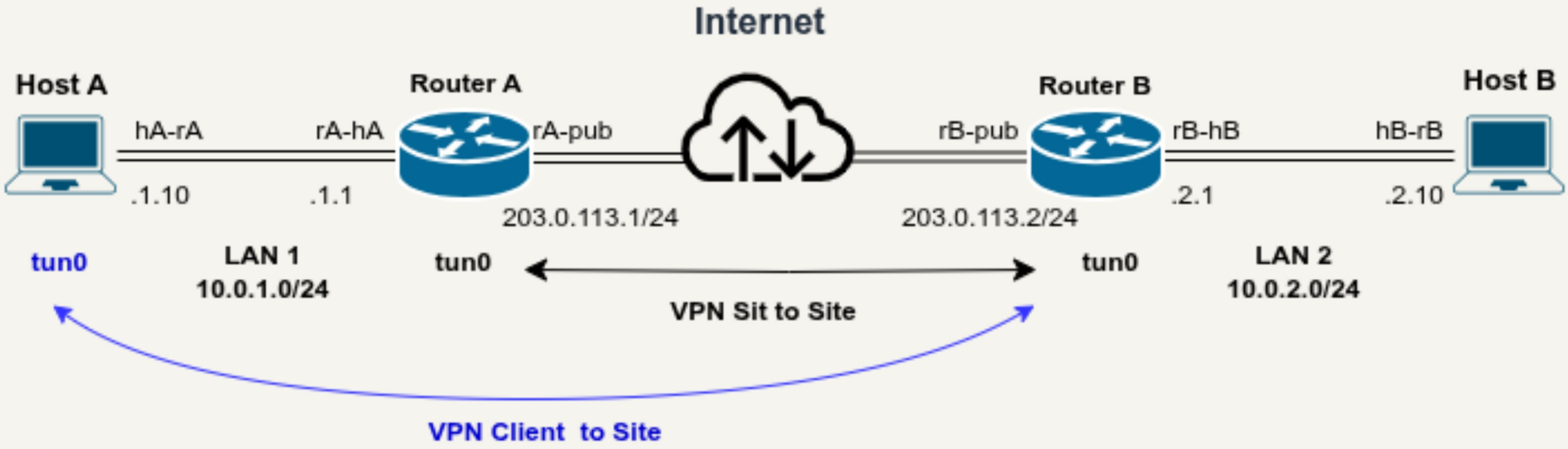
Paolo Santinelli



IL PROBLEMA

Due sedi, due reti private, nessun percorso

Una VPN (Virtual Privat Networl) fa si che reti e dispositivi lontani si comportano come se fossero sulla stessa rete locale, usando Internet come collegamento. Di solito la VPN protegge anche il traffico, ma non è scontato: lo vedremo.



Perché non funziona (senza VPN)

Gli indirizzi privati (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16) non vengono instradati dai router di Internet

CONCETTI DI BASE

Cos’è una VPN: una busta dentro un’altra

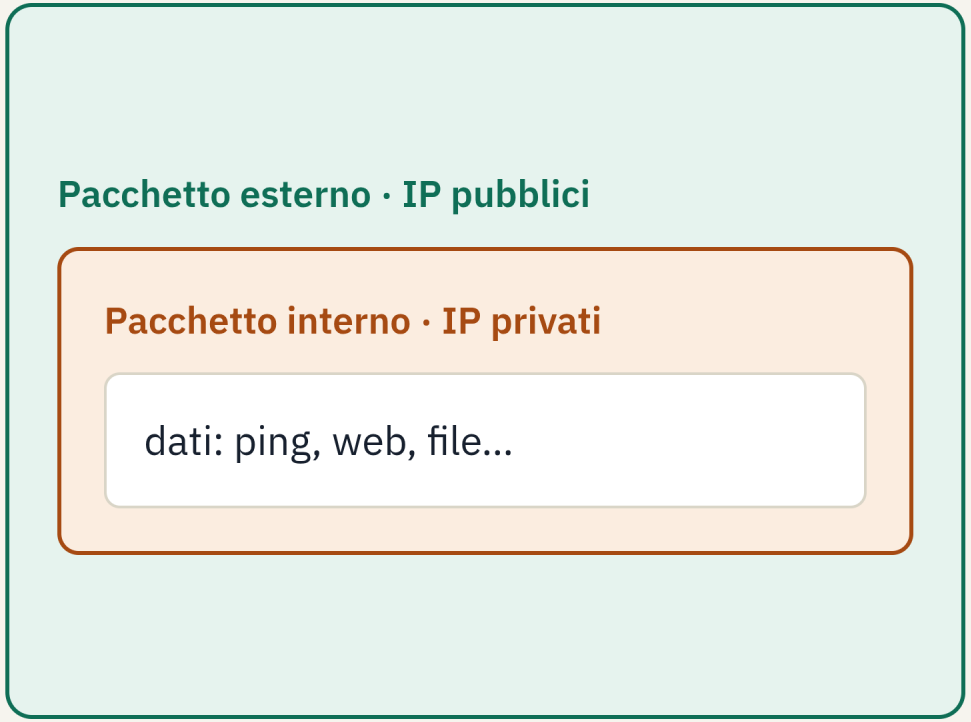
Tunneling

Un pacchetto viene inserito dentro un altro pacchetto, con indirizzi diversi.

Cifratura

Il contenuto diventa illeggibile per chi sta in mezzo.

Sono due cose distinte: una VPN può avere l’una, l’altra o entrambe.



CONCETTI DI BASE

I tipi principali di VPN

SCENARIO

Site-to-site

Collega due reti intere, da router a router. Gli host non sanno nulla della VPN. È lo scenario della nostra dimostrazione.

SCENARIO

Client-to-sinte (remote access)

Un singolo dispositivo, come un portatile o un telefono, si collega alla rete dell’azienda o della scuola.

LIVELLO

Livello 3

Trasporta pacchetti IP. GRE, IPIP, IPsec, WireGuard, OpenVPN in modalità tun.

LIVELLO

Livello 2

Trasporta frame Ethernet, come un cavo esteso. GRETAP, OpenVPN in modalità tap.

PROTOCOLLI

Quattro modi di costruire un tunnel

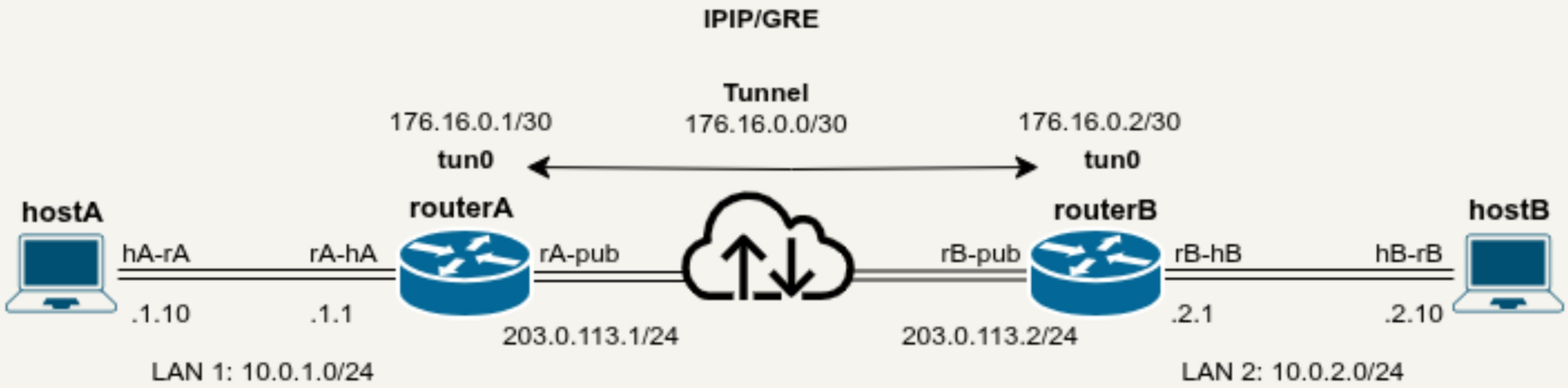
ESISTONO ALTRI TIPI DI VPN

Protocollo	Cifratura	Dove gira	In breve
GRE / IPIP	Nessuna	Kernel	Incapsulamento puro (usata per finalità didattiche)
IPsec	Sì (ESP)	Kernel e demone IKE	Standard, flessibile, complesso
OpenVPN	Sì (TLS)	Userspace	Flessibile, passa anche su TCP
WireGuard	Sì, sempre	Kernel	Codice compatto, un solo cifrario

In verde le righe che vedremo nella dimostrazione.

PROTOCOLLI

GRE e IPIP: solo incapsulamento



PROTOCOLLI

GRE e IPIP: solo incapsulamento

IPIP

`IP dentro IP · protocollo 4`



Il più semplice: il pacchetto originale diventa il carico di un nuovo pacchetto IP. Aggiunge 20 byte (header pacchetto esterno) al pacchetto IP originale (interno) e trasporta solo traffico IP.

GRE

`Incapsulamento generico · prot. 47`



Più flessibile: un piccolo header dice cosa c'è dentro (IPv4, IPv6, perfino frame Ethernet con GRETAP). Aggiunge 24 byte.

Nessuno dei due protocolli cifra o autentica: chi vede il pacchetto lo legge, e può anche falsificarlo.

DIMOSTRAZIONE · IL LABORATORIO

Network namespace: tante reti su un solo PC



Network Namespace

Uno stack di rete isolato, con interfacce, rotte e firewall propri.

Coppia veth

Un cavo virtuale: ciò che entra da un capo esce dall'altro.

Segmento pubblico

203.0.113.0/24, il collegamento arancione: qui catturiamo il traffico.

Namespace

I namespace sono un meccanismo del kernel che permette di isolare le risorse di sistema, facendo sì che processi diversi possano avere una visione differente dello stesso sistema operativo.

DEMO 1 · SENZA CIFRATURA

Prima della VPN: i due siti non si vedono

```
$ sudo ./vpn-lab-up.sh
$ sudo ./vpn-lab-test.sh
[ OK ] hostA → routerA
[ OK ] routerA → routerB
[ OK ] hostB → routerB
[FALLITO] hostA → hostB
```

Funzionano

Tutti i collegamenti diretti tra macchine vicine.

Non funziona

Da Host A a Host B: nessun router conosce la rete 10.0.2.0/24.

DEMO · ESPLORARE IL LABORATORIO

Dentro un namespace: i comandi utili

```
# guardare
$ ip netns list
$ ip -n routerA -br addr
$ ip -n routerA route
$ ip -n routerA route get 10.0.2.10
# eseguire
$ sudo ip netns exec hostA ping -c3 10.0.2.10
$ sudo ip netns exec routerA bash
# osservare
$ sudo ip netns exec routerA tcpdump -ni rA-pub
```

Namespace	Interfacce
hostA	hA-rA
routerA	rA-hA, rA-pub
routerB	rB-pub, rB-hB
hostB	hB-rB

Dopo il tunnel

Sui router compare tun0 (IPIP o GRE) oppure wg0 (WireGuard).

ip -n NOME è la scorciatoia per ip netns exec NOME ip.

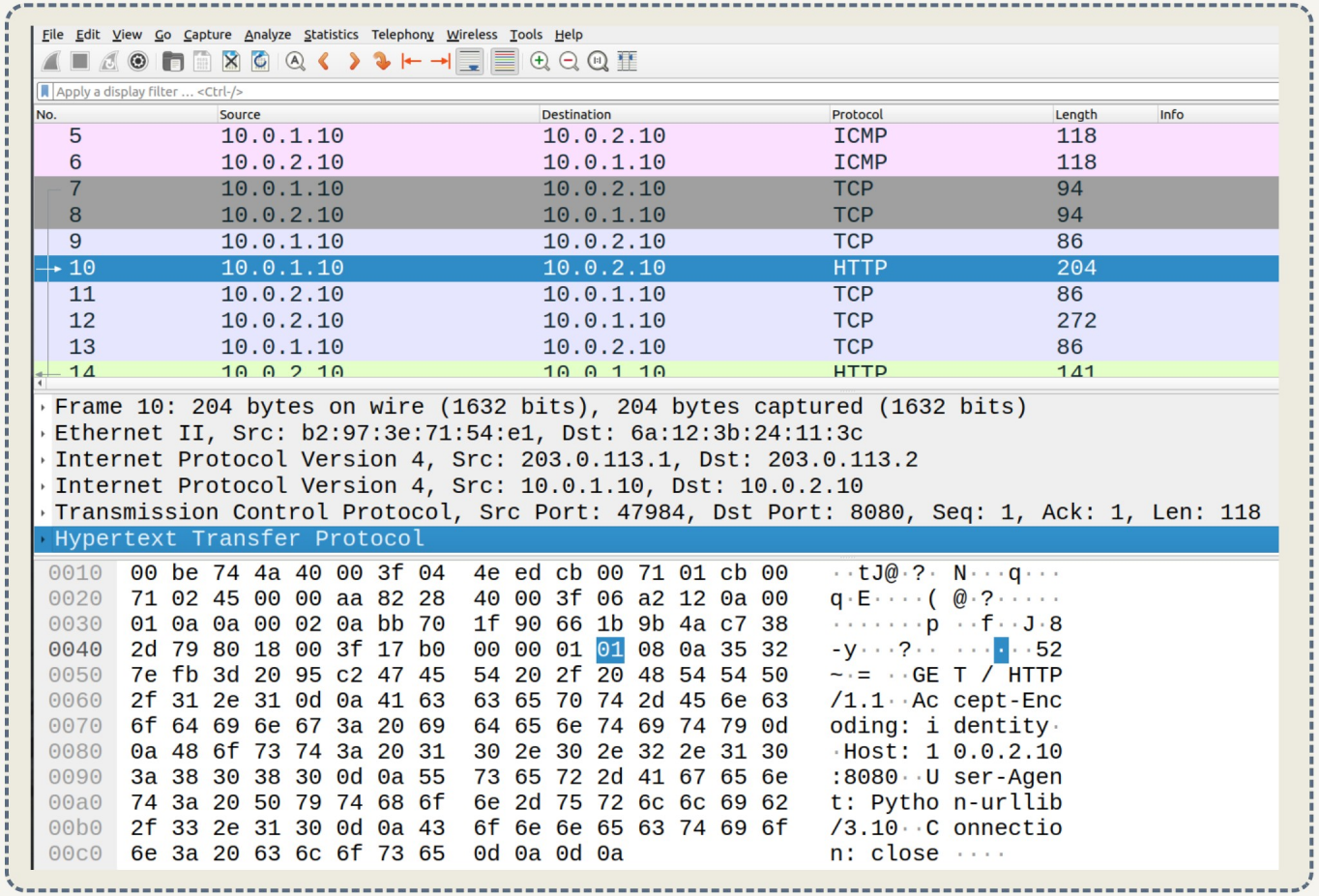
DEMO 1 · SENZA CIFRATURA · IPIP

Il tunnel IPIP: due header IP impilati

```
$ sudo ./vpn-lab-tunnel.sh ipip
$ sudo ip netns exec routerA \
tcpdump -i rA-pub -nn -v
$ sudo ip netns exec hostA \
ping 10.0.2.10
```

cattura su rA-pub: protocollo IPIP (4)
esterno 203.0.113.1 → 203.0.113.2
interno 10.0.1.10 → 10.0.2.10

Sul segmento pubblico il contenuto si legge in chiaro.



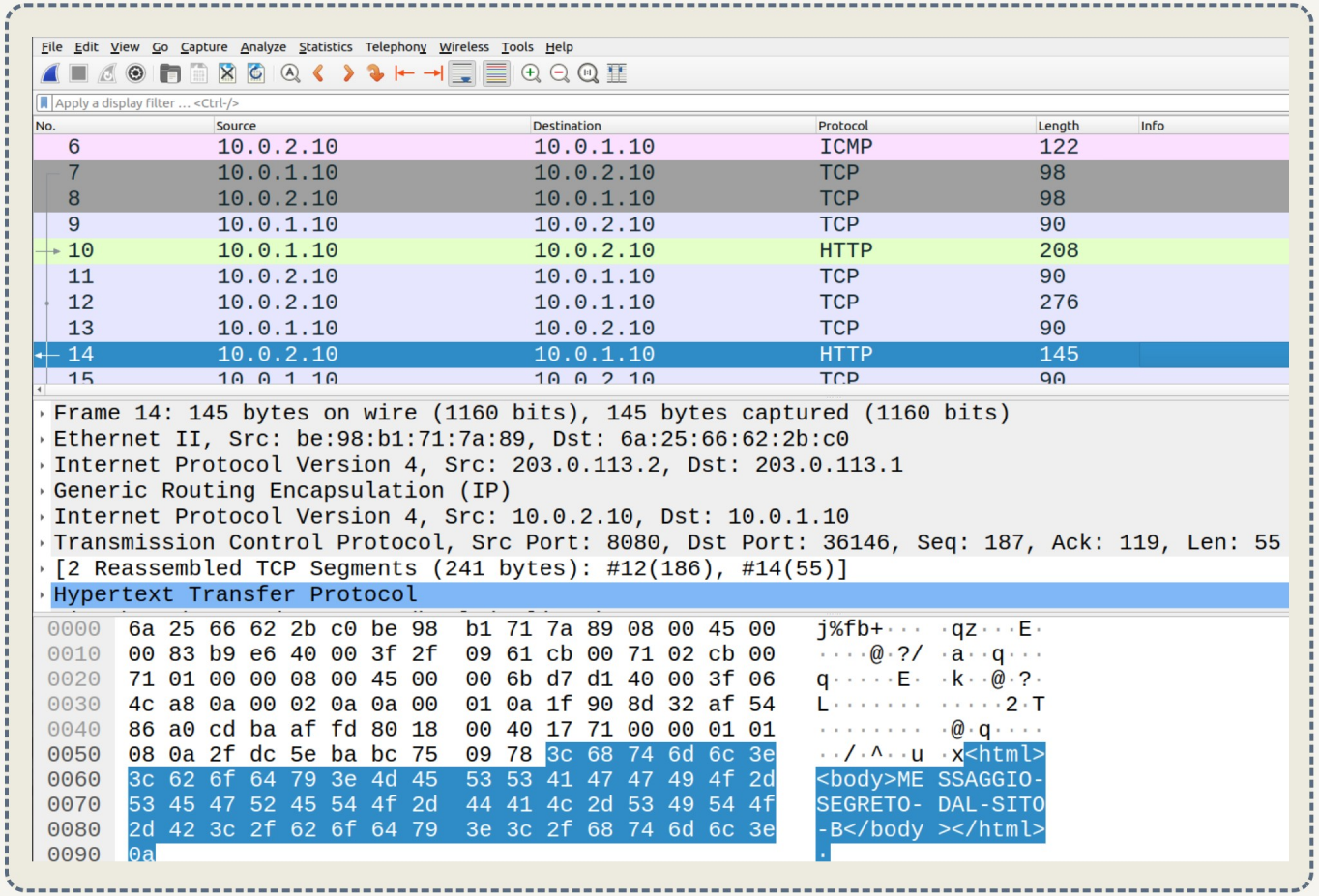
DEMO 1 · SENZA CIFRATURA · GRE

Il tunnel GRE: header IP - GRE - IP impilati

```
$ sudo ./vpn-lab-tunnel.sh gre
$ sudo ip netns exec routerA \
tcpdump -i rA-pub -nn -v
$ sudo ip netns exec hostA \
ping 10.0.2.10
```

cattura su rA-pub: protocollo GRE (47)
esterno 203.0.113.1 → 203.0.113.2
interno 10.0.1.10 → 10.0.2.10

Sul segmento pubblico il contenuto si legge in chiaro.



WIREGUARD · CARATTERISTICHE

WireGuard: poco codice, poche scelte

Nel kernel, compatto

Integrato in Linux dalla versione 5.6. Poche migliaia di righe di codice: più facile da leggere e da verificare.

Un solo insieme di cifrari

ChaCha20-Poly1305, Curve25519, BLAKE2s. Niente negoziazione, quindi niente configurazioni deboli da scegliere per errore.

Chiavi, non certificati

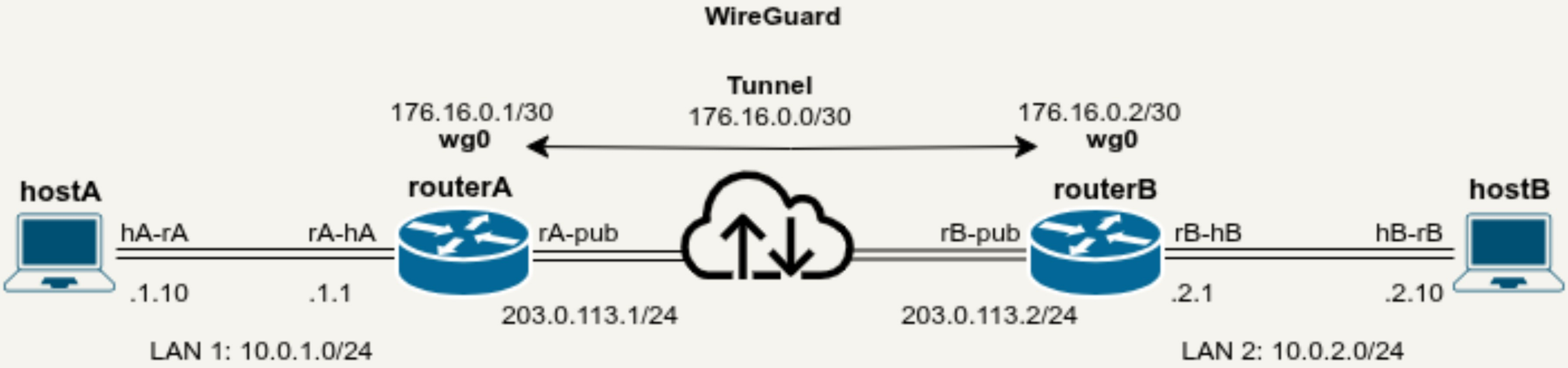
Ogni peer ha una coppia di chiavi. La chiave pubblica è la sua identità, come in SSH.

UDP e discreto

Di solito porta UDP 51820. Non risponde a chi non conosce la chiave giusta e, se non c'è traffico, non invia nulla.

Stesso protocollo su Linux, Windows, macOS, Android e iOS.

WireGuard: poco codice, poche scelte



Stesso protocollo su Linux, Windows, macOS, Android e iOS.

DEMO 2 · CON CIFRATURA

WireGuard: la chiave pubblica è l’identità

```
$ sudo ./vpn-lab-wireguard.sh
$ sudo ip netns exec routerA \
wg show
```

Chiavi

Una coppia per router, con wg genkey e wg pubkey. Niente certificati.

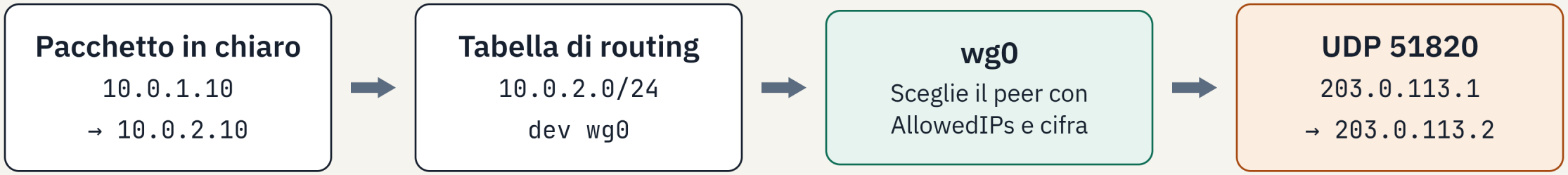
AllowedIPs

A quale peer cifrare in uscita, e da quali indirizzi accettare in entrata.

L’handshake parte al primo pacchetto, non alla creazione del tunnel.

WIREGUARD · INTERFACCIA E TUNNEL

wg0 non è il tunnel: ne è l'ingresso



wg0, l'interfaccia

Un'interfaccia virtuale del sistema: ha un nome e un indirizzo (172.16.0.x), compare in ip a e si cattura con tcpdump. Qui il traffico è in chiaro.

Il tunnel, il percorso

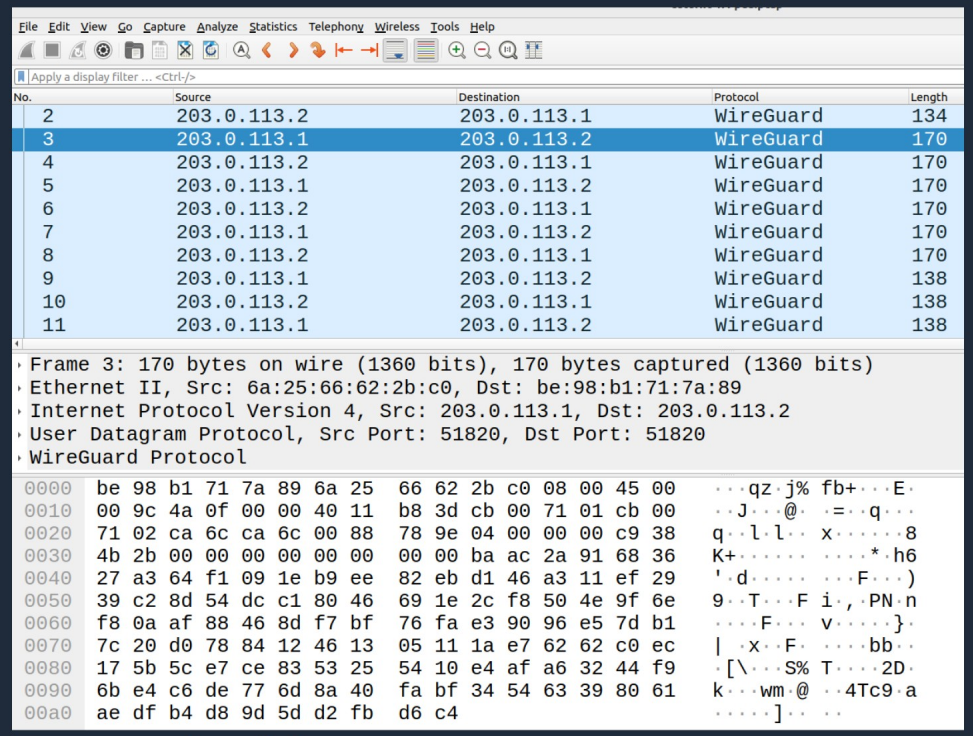
Non è un oggetto: sono i pacchetti UDP cifrati tra i due indirizzi pubblici. Si vedono su rA-pub, e lì la frase non si legge.

Lo stesso schema di tun0 con IPIP e GRE

DEMO 2 · IL CONFRONTO

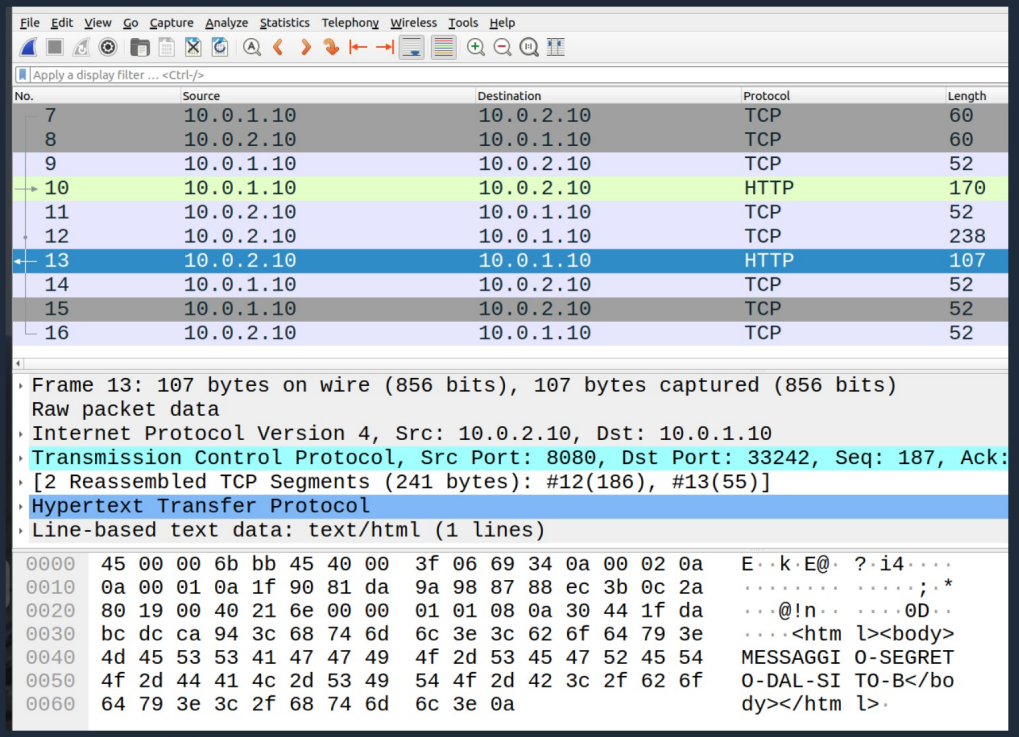
La stessa frase, due punti di osservazione

rA-pub · segmento pubblico



Frase non leggibile

wg0 · dentro il tunnel



Frase in chiaro

Non è rumore puro: su rA-pub restano visibili il tipo di messaggio, gli indici e i contatori. Cifrati sono l’identità e il contenuto.

RIEPILOGO

Cosa abbiamo visto sui pacchetti

	GRE / IPIP	WireGuard
Su rA-pub	Header visibili, dati in chiaro	UDP 51820, contenuto cifrato
Su tun0 / wg0	Pacchetto originale	Pacchetto originale, identico
Autenticazione	Nessuna	Chiavi pubbliche
Overhead	20 byte (IPIP), 24 (GRE)	circa 60 byte su IPv4

Il tunnel risolve la raggiungibilità, la cifratura risolve la riservatezza: sono due problemi diversi.

WIREGUARD · DA PROVARE A CASA

Creare la propria VPN WireGuard

Linux

`sudo apt install wireguard`

File /etc/wireguard/wg0.conf, poi wg-quick up wg0.

wireguard.com/quickstart

Windows

`App WireGuard`

Installer ufficiale. “Add empty tunnel” genera la coppia di chiavi.

wireguard.com/install

Android

`App su Google Play`

Importa un file .conf o scansiona un QR code generato sul server.

wireguard.com/install

Su ogni piattaforma i passi sono gli stessi

1. Genera una coppia di chiavi su ciascun dispositivo.
2. Scambia le chiavi pubbliche.
3. Imposta Endpoint e AllowedIPs.
4. Lato server, raggiungibile la porta UDP 51820.

PER CONTINUARE

Domande?

Da provare a casa

Gli script del laboratorio:

github.com/paolo-santinelli/Learning_VPN

Comandi per esplorare: ip netns, ip -n, wg show

Per approfondire

wireguard.com (quickstart, install, protocol) · docs.strongswan.org · openvpn.net

RFC 2003 (IPIP), RFC 2784 (GRE), RFC 4303 (ESP)

Scansiona per aprire GitHub

Paolo Santinelli · santinelli.paolo@gmail.com